

listing access specifiers and explain.

Access specifiers are also called access modifiers which define the visibility and accessibility of classes, methods, variables and Constructors.

There are four access specifiers in Java.

1. public

2. private

3. protected

4. default (no keyword).

1. public

* Declared using that public keyword.

* Can be accessed from anywhere in the program.

* provides the highest accessibility.

Ex:- public int marks = 90;

2. Private

* Declared using the private keyword.

* Can be accessed only within the same class.

* used to achieve data binding and encapsulation

Ex:- private int age = 20;

3. protected:-

* Declared using the protected keyword.

* Accessible within the same package.

* Also accessible in subclass including subclasses in other packages.

Ex:- protected int salary = 30000;

4. default:-

* No keyword is used.

* Accessible only within the same package.

* It is also called package-private, access.

Ex:- `int rollNo = 101;`

b

Describe class members for box object.

A class member is a variable or method declared inside a class. For a Box object, the class can contain data members and member methods.

Example: Box class

```
class Box {
```

```
    double length;
```

```
    double width;
```

```
    double height;
```

```
    void setDimensions(double l, double w, double h) {
```

```
        length = l;
```

```
        width = w;
```

```
        height = h;
```

```
    }
```

```
    double volume() {
```

```
        return length * width * height;
```

```
    }
```

```
}
```

Description of class members

1. Data members

- * length, width and height are data members.
- * They store the dimensions of the box.
- * Their values are associated with each Box object.

2. member methods

- * `setDimensions()` assigns values to the dimensions.
- * `Volume()` calculates and returns the volume of the box.

3. object

→ A object of the Box class can be created as:

```
Box b1 = new Box();
```

4. Accessing members

Members can be accessed using dot (.) operator.

```
b1.length = 10;
```

```
b1.Width = 5;
```

```
b1.height = 4;
```

```
System.out.println(b1.Volume());
```

Formula:

Volume of Box = length $\times$ width $\times$ height

For length = 10, width = 5, height = 4;

Volume = $10 \times 5 \times 4 = 200$ Cubic units.

Box class program in JAVA

```
class Box {
```

```
    double length;
```

```
    double width;
```

```

double height;
double volume() {
    return length * width * height;
}

public static void main (String[] args) {
    Box b1 = new Box();
    b1.length = 10;
    b1.width = 5;
    b1.height = 4;

    System.out.println ("length = " + b1.length);
    System.out.println ("width = " + b1.width);
    System.out.println ("height = " + b1.height);
    System.out.println ("volume of Box = " + b1.volume());
}
}

```

Output :-

length = 10.0

width = 5.0

height = 4.0

Volume of Box = 200.0

2.
a

Differentiate final classes with classes.

Normal class	Final class
* Declared using class keyword * Can be extended by another class * Supports inheritance * Its methods can be overridden by subclasses. * Can act as a parent / superclass. * Useful when customization through inheritance is required. * Provides flexibility for creating subclasses. * Subclasses can add new features to it. * Example :- Class vehicle <pre> class vehicle { // ... } </pre> * Class B extends A is possible	* Declared using final class. * Cannot be extended by another class. * Does not support inheritance * No subclass exists to override its methods. * Cannot act as a parent / superclass. * Useful when inheritance must be prevented. * provides restrictions on creating subclasses. * New features final class cannot be added through inheritance. * Example :- final class vehicle <pre> final class vehicle { // ... } </pre> * Class B extends A is not possible if A is final.

b Demonstrate the use of `this` reference in Java.

The `this` keyword is a reference variable in Java that refers to the current object.

Main uses of `this`

1. To refer to current class instance variables
2. To call current class methods.
3. To call the current class constructor
4. To pass the current object as an argument.
5. To return the current object.

Example: program

```
class Student {
```

```
    int rollNo;
```

```
    String name;
```

```
    Student (int rollNo, String name) {
```

```
    {
```

```
        this.rollNo = rollNo;
```

```
        this.name = name;
```

```
    }
```

```
    void display() {
```

```
        System.out.println("Roll No: " + this.rollNo);
```

```
        System.out.println("Name: " + this.name);
```

```
    }
```

```
    public static void main (String[] args) {
```

```
        Student s1 = new Student (101, "Likhitha");
```

```
        s1.display();
```

```
    }
```

output:-

Roll No: 101

Name: Likhitha.

3a. How overloading is differ with over riding.

Method Overloading	Method over riding
* Same method name with different parameters. * Inheritance not required. * Parameters must be different. * Return-type may be same or different, but cannot differ only by return-type. * Class usually occurs within the same class. * Binding Compile-time. * Purpose provides different ways to perform an operation. * Access modifier no special restriction. * Static methods can be over loaded. * Example:- <code>add(int int)</code> and <code>add(int int, int)</code>	* Subclass provides a new implementation of a parent method. * Inheritance is required. * Parameters must be the same. * Return-type same or compatible return-type. class occurs between parent * Binding Runtime and child class * Binding Run-time. * Purpose changes the behavior inherited from the parent class. * Access modifier cannot reduce the parent's method accessibility * Static methods are not truly over ridden; they are hidden. * Example:- <code>Animal.sound()</code> and <code>Dog.sound()</code>

Q MARKS:-

1. Define object. Give any three examples of objects.

Objects are basic run-time entities in an objects oriented systems (oop). Any real world entity is called an object.

Examples:-

1. Student - `Student s1 = new Student();`
2. Car `C1 = new Car();`
3. Bank Account `ba1 = new Bank Account();`

2. Write the syntax of create object and take one example create object.

Class Name Object Name = `new class Name();`

Ex:- `Student s1 = new Student();`

Here, student → class name

s1 → Object name

new → keyword used to create an object

student() → Constructor.

3. list out access specifiers in java.

There are four access specifiers in java.

1. private

2. default

3. public

4. protected

4. Write the necessity of this reference.

* To distinguish between instance variables and local variables with the same name.

- * To access the current object's variables and methods.
- * To invoke the current class constructor using `this()`.
- * To return the current object from a method.

5 What is overloading?

Overloading is a feature in Java in which two or more methods have the same name but different parameters in the same class.

Example:- Class Calculator {

int add (int a, int b) {

return a+b;

}

int add (int a, int b, int c) {

return a+b+c;

}

}

6 Can you overload Constructors? explain.

Yes, Constructors can be overloaded in Java.

Constructor overloading means having more than one constructor in the same class with different parameters.

Example:-

class Student {

Student() {

System.out.println ("Default Constructor");

}

Student (String name) {

System.out.println ("Name: " + name);

}

Student (String name, int age) {

```
System.out.println("Name: " + name + ", Age: " + age);
```

```
}
```

```
public static void main (String[] args)
```

```
{  
    Student s1 = new student ();
```

```
    Student s2 = new student ("Likhitha");
```

```
    Student s3 = new student ("Likhitha", 18);
```

```
}
```

```
}
```

Output:-

Default Constructor

Name : Likhitha

Name : Likhitha , Age : 18

7 The object have width, height, length, and its also have the volume. Develop a class for above Object.

Class Box {

double width;

double height;

double length;

double volume() {

return width * height * length;

}

public static void main (String[] args) {

Box b1 = new Box();

b1.width = 10;

b1.height = 5;

b1.length = 4;

```
System.out.println("width = "+ b1.width);  
System.out.println("Height = "+ b1.height);  
System.out.println("length = "+ b1.length);  
System.out.println("volume = "+ b1.volume);  
}  
}
```

Output :-

width = 10.0

Height = 5.0

length = 4.0

volume = 200.0

